# 长上下文技术 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉 Transformer 和注意力机制的基本概念，可以直接翻阅主报告。

---

## 一、什么是"上下文长度"?

上下文长度（Context Length）指的是大模型**一次能"看到"多少文本**。

直观理解：你给 ChatGPT 粘贴了一篇 5 万字的论文，问它"论文第三章的结论是什么"。模型能不能回答，取决于它的上下文长度是否 ≥ 5 万字。如果模型只支持 4 万字，它根本"看不到"第三章的内容。

### 为什么上下文长度很重要?

- **读长文档**：分析合同、论文、代码库
- **多轮对话**：Agent 跟用户聊了 50 轮，要记住前面说了什么
- **跨文件推理**：看了一个项目的所有代码文件再回答问题

### 上下文长度的"水分"

一个残酷的事实：**模型宣称的上下文长度 ≠ 它能有效使用的上下文长度。**

就像你说"我能背 1000 页书"，但距离你 200 页位置那句话你只能记得 3 成。模型也一样——OpenAI 的 GPT-4o 宣称 128K 上下文，但在长距离检索测试中，中间位置的内容准确率会大幅下降。

这就是著名的 **Lost in the Middle（迷失在中间）** 现象。

---

## 二、位置编码——模型怎么知道词语的顺序?

"我打你"和"你打我"词语一样，意思完全相反。模型必须知道词语的顺序。

### 最初的方案：绝对位置编码

2017 年 Transformer 诞生时，给每个位置分配一个"座位号"：第 1 个词加编码 1，第 2 个词加编码 2……

**痛点**：模型训练时只见过 1–1024 号的座位。你让它处理 2000 个词，第 1500 号座位的编码它从没见过，表现自然一塌糊涂。

### RoPE（Rotary Position Embedding，旋转位置编码，2021）

现在最主流的位置编码方案（被 Llama、Mistral、Qwen、GPT-4o、Claude 等几乎所有主流模型采用）。

**直觉**：想象你在一个旋转餐厅里——你坐的位置（语义信息）不变，但餐桌在转（位置信息用旋转角度表示）。你跟你旁边的人说话（短距离），跟 50 米外的人喊话（长距离），难度自然不同。

RoPE 把 query 和 key 向量做"旋转"，旋转量取决于它们的位置。**两个词越近，它们的旋转量之差越小**，注意力分数越高。这个设计天然支持相对位置感知。

比喻不完美但有用：**RoPE 像给每个词发了一把刻度尺，尺子上标着"离我有多远"**。

### ALiBi：给远距离词"扣分"（2021）

比 RoPE 更简单粗暴：不给位置编码，直接在注意力分数上按距离扣分——距离越远扣分越多。

思路像**一个不耐烦的读者，越往后看越没耐心**。好处是天然支持超长文本（因为不依赖训练时的位置），但远距离信息捕捉能力不如 RoPE。

---

## 三、为什么长文本这么耗显存?

训练大模型时，显存被三样东西占着：模型参数、优化器状态、**中间激活值**（每层的输出）。

对于长文本，中间激活值是显存黑洞。因为注意力计算的核心操作：

> 每个词要看所有词 → 产生一个 N×N 的注意力矩阵（N 是文本长度）

序列长度翻倍，注意力矩阵的大小翻 **4 倍**（$N^2$）。处理 128K 文本的注意力矩阵是 4K 文本的 **1024 倍**。

**这就是长上下文技术要解决的核心矛盾：文本越长，计算量和显存需求爆炸式增长。**

---

## 四、FlashAttention——让长文本变得可行

### 问题出在哪?

GPU 有高速缓存（SRAM，又快又小）和显存（HBM，慢且大）。标准 attention 需要反复读写显存——就像**频繁从冰箱取菜而不是用手边的备菜台**。

### FlashAttention 的洞察

把注意力计算切成小块，在高速缓存里一块一块算完，最后拼起来。**显存读写次数从 $O(N^2)$ 降到 $O(N)$**。

打个比方：你不再需要把所有人的聊天记录全部写在黑板上再分析，而是分组聊天，每组聊完把结论记下来。

**FlashAttention 是所有长上下文技术的基础设施**——没有它，128K+ 的上下文在工程上根本不可行。

---

## 五、KV Cache——推理时的显存瓶颈

### 推理时发生了什么？

模型生成文本是一个词一个词蹦的。生成第 100 个词时，前 99 个词的 Key 和 Value 向量已经算过了——但如果不缓存，每次都要从头重算。

**KV Cache** 就是把前面所有词的 K/V 向量存下来，避免重复计算。

### 问题

上下文长度每翻一倍，KV Cache 也翻一倍。128K 上下文的 KV Cache 占用可达数 GB。

打个比方：**你在写一本 100 页的书，写完第 50 页后，前面 50 页的"摘要"占了半个书架。写到第 100 页时，书架满了。**

### 解决方案

| 方案 | 思路 | 类比 |
| --- | --- | --- |
| MQA（Multi-Query Attention，多查询注意力） | 所有注意力头共享同一组 K/V | 多个厨师共用一个备菜台 |
| GQA（Grouped Query Attention，分组查询注意力） | 分组共享 K/V | 每 4 个厨师共用一个备菜台 |
| MLA | 把 K/V 压缩成"浓缩液"，用时再还原 | 把蔬菜打成浓缩汁，做菜时再还原 |

MLA（DeepSeek 的核心创新）是目前最强的方案——把 KV Cache 压缩了 93%–98%。

---

## 六、上下文扩展——让模型支持更长的文本

模型训练时只用 4K 长度的文本，怎么能让它处理 128K？

### 线性插值（PI）

把 128K 个位置"压缩"映射到模型熟悉的 4K 个位置上。类似**把一张大照片缩小到相框里**——信息都在，但分辨率降低了。

### YaRN

更聪明的方法：不简单压缩，而是调整 RoPE 的旋转频率。**高频部分（细粒度位置）少缩放，低频部分（粗略位置）多缩放。**

就像调收音机——高频台（精确定位）微调即可，低频台（大致方向）大调。

### NTK-aware scaling

跟 YaRN 类似，基于神经正切核理论，也是"不同频率不同缩放"。被社区广泛采用。

---

## 七、StreamingLLM——处理无限长文本

如果文本无限长（比如直播字幕），怎么办？

研究者发现了一个神奇现象：**Attention Sink（注意力沉没）**。

模型对第一个 token（不论内容是什么）会分配异常高的注意力分数。原因是 softmax 要求所有注意力之和为 1，模型把"多余的注意力"倾倒给了第一个 token。

**解决方案**：只保留开头几个 token（作为注意力"垃圾桶"）+ 最近的若干 token。中间的统统丢掉。

代价：**它只能"看到"最近的文本**，中间内容完全丢失。适合实时字幕等场景，不适合需要精确回忆全文的任务。

---

## 八、位置编码的"外推"——为什么模型能处理没见过的长度

外推 = Extension = 模型训练时最长只见过 4K 文本，但推理时能处理 128K"推"得更远。

### 三条路线

1. **位置编码天生好**：ALiBi 天然支持外推
2. **扩展方法（PI/YaRN/NTK）**：通过缩放/旋转调整适配新长度
3. **训练时渐进**：先在短文本上预训练，再在长文本上继续训练

大多数模型走的是 **2+3 组合**路线。

---

## 九、评估——怎么知道模型真的"看懂了"长文本？

### Needle in a Haystack（大海捞针）

在超长文本（干草堆）里藏一句"验证码是 8472"，问模型"验证码是多少"。这是最流行的长上下文评测方法。

**局限**：模型只要找到字面匹配就能答对，不需要真正理解。

### NoLiMa（更高难度的评测）

不藏简单密码，而是藏**语义信息**，模型必须真正理解才能答对。结果：

| 模型 | 宣称 | 有效（NoLiMa） |
|---|---|---|
| Llama 4 Scout | 10M | ~1K |
| GPT-4.1 | 1M | ~16K |
| Claude 3.5 Sonnet | 200K | ~4K |

**宣称和有效差距巨大**——这是当前长上下文领域最重要的认知。

---

## 十、长上下文 vs RAG——不是二选一

RAG（检索增强生成）是把外部文档切碎、索引、只搜相关片段给模型看。

| 方案 | 优势 | 劣势 |
|---|---|---|
| 长上下文 | 一次给模型看全文 → 全局理解更好 | 贵、慢、有效利用率低 |
| RAG | 便宜、快、可扩展 | 可能遗漏信息、碎片化 |

2026 年的共识：**不用二选一，而是混合架构**——用 RAG 召回相关段落，用长上下文做综合分析。

---

## 十一、一张图总结

```
问题：文本变长 → 注意力计算 O(N²) → 显存爆炸
            ↓
怎么解决?
            ↓
┌──────────────────────────────────────────────────────────────┐
│  并行技术路线                                                  │
│                                                               │
│  位置编码 ← RoPE/YaRN/ALiBi ← 决定模型能"推"到多长            │
│  计算优化 ← FlashAttention ← 让长文本训练在工程上可行          │
│  显存压缩 ← MQA/GQA/MLA ← 减少 KV Cache 占用的 90%+           │
│  分布式   ← Ring Attention ← 多卡分摊长序列计算                │
│  推理优化 ← StreamingLLM/H2O ← 动态丢弃不重要的缓存            │
│  评测     ← NIAH → RULER → NoLiMa ← 识别"虚假上下文"           │
└──────────────────────────────────────────────────────────────┘
            ↓
结果：2017 年 512 tokens → 2026 年 1M+ tokens
      但"宣称"和"有效"之间的鸿沟仍是核心挑战
```